Semantic Aware AI Scheduler

Satya Srinivas Paladugu [BL.AI.U4AID24063]
Nandini Padavala [BL.SC.U4CSE24133]
Jahnvi KV [BL.SC.U4ECE24034]

Abstract

Multiple devices deliver image messages to a receiver with a single nonpreemptive decoder. A receiver-side scheduler selects the next complete message for reconstruction. This project studies whether information about image importance, mission relevance, repetition and visible scene content can make that decision better. Each complete message contains semantic scheduling metadata and an encoded image payload. Selection uses the metadata and causal receiver context; reconstruction uses the selected payload.

The work includes parameter extraction, four-class image-priority classification, a 6000-image dataset, simulated device streams and evaluation of eight rule-based policies and fourteen learned schedulers. On the matched development workload, FIFO recorded an average waiting penalty of 5.8220, SPI-first recorded 3.3741 and LambdaMART recorded 3.2183. Lower penalty means less costly waiting under the declared test rules. LambdaMART was the best of the methods compared, with 44.72 percent lower penalty than FIFO and 4.62 percent lower penalty than SPI-first. These are simulation results using model-generated priority references. All queue timings are expressed in ms, with a constant-service control of 1 ms per message; this is a simulator calibration, not measured SQ-GAN decoding latency.

Project goal

The goal is to show important and useful images to the receiver earlier. For example, a routine photograph should not automatically be processed before an image of a serious incident merely because it arrived first. The scheduler must also account for images left waiting. It cannot claim success by ignoring messages it never selects.

1. Header and latent payload architecture

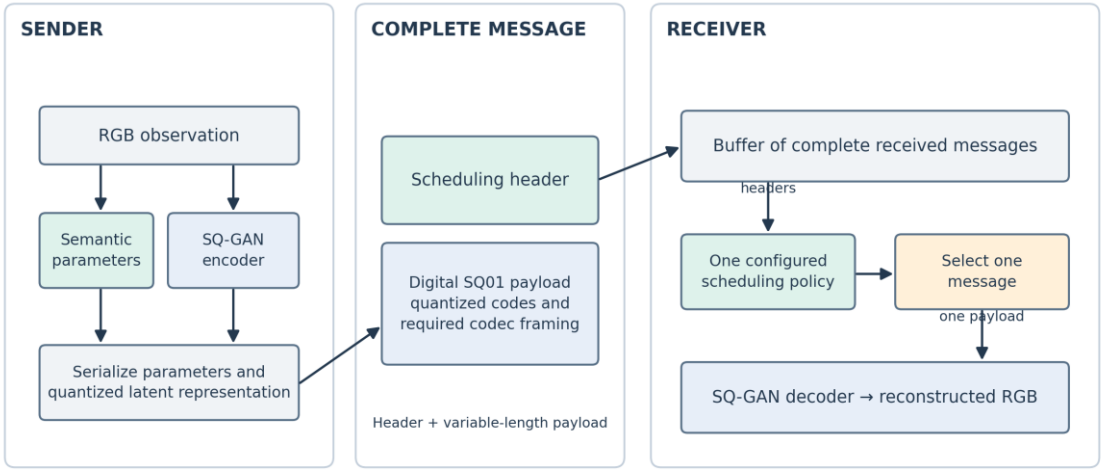


Fig. 1. Sender and receiver architecture with SQ-GAN as the digital codec. Parameter extraction and latent encoding operate at the sender; reconstruction follows receiver-side selection.

What the header contains

The scheduling header carries priority, mission relevance, appearance novelty, scene measurements, availability, identity and timing. The scheduler reads this metadata before selecting a payload for reconstruction.

What the latent payload contains

We selected SQ-GAN as the digital neural source codec [21]. It maps image features to discrete codebook entries through vector quantization, allowing the encoded representation to be serialized as a digital payload. Semantic masking allocates representation capacity to task-relevant content. This is a better architectural fit for the project's byte-buffered receiver interface than treating analog channel symbols as an ordinary packet.

The SQ-GAN paper reports perceptual and semantic-segmentation advantages over comparison codecs at very low bit rates [21]. Those published results motivate the choice; they are not a local, rate-matched codec benchmark on our scheduler dataset. Quantization makes the representation digital but does not make the reconstructed image pixel-exact.

The repository contains an SQ-GAN digital adapter with native 256 by 512 RGB input and output. The transmitter obtains the paired semantic map, encodes the representation and produces SQ01 payload bytes. The receiver decodes those bytes without the original RGB image or transmitter map. SQ01 is the project's experimental packet framing, not the paper's entropy coder. Checkpoints and shared codebooks are setup assets rather than per-image header fields.

Unlike direct analog JSCC mappings [1], SQ-GAN provides discrete source representations suitable for byte-oriented message storage. The payload adapter still defines codec identity, tensor shape and framing.

2. Receiver buffer and one-message dispatch

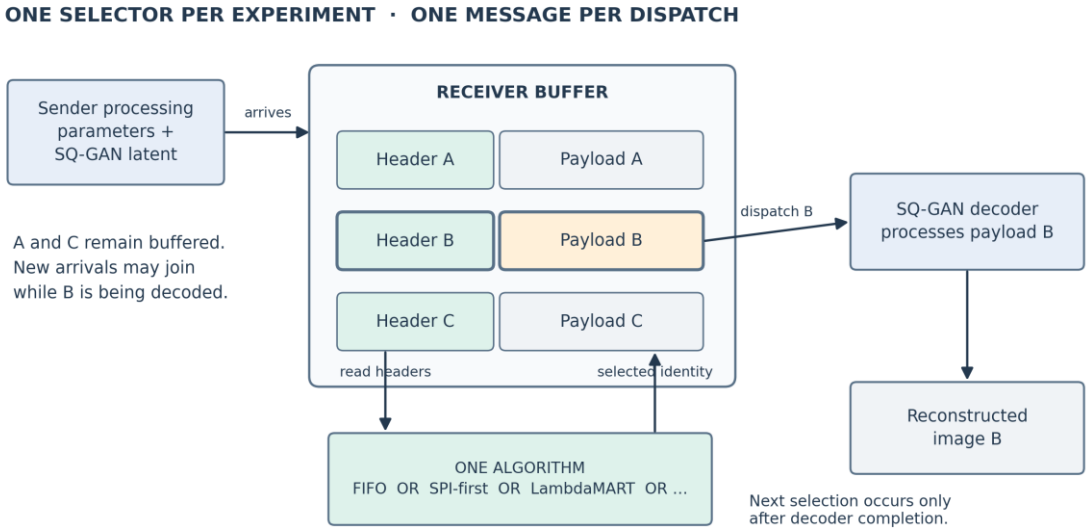


Fig. 2. Message flow at the receiver. Complete header-plus-payload messages enter the buffer. One configured scheduling algorithm scores eligible headers and dispatches exactly one selected payload to SQ-GAN decoding.

The transmitter produces two outputs for each observation: scheduling parameters and an encoded latent payload. They are assembled into one complete message. The receiver admits the message to its buffer only after the message is complete and compatible with the declared codec profile.

When the decoder is idle, the receiver invokes one selected scheduling policy, such as FIFO, SPI-first, RankNet or LambdaMART. It does not combine all policies into one decision. The policy evaluates currently buffered candidates and returns one message identity. The receiver removes that message from the ready set and sends its payload to the decoder. Other messages remain buffered, and new arrivals may join during processing.

Dispatch is nonpreemptive and one-shot. A new Critical message cannot interrupt the payload already being decoded, and the scheduler does not create an application-level retransmission. The receiver updates its lifecycle records after completion and then takes a fresh queue snapshot. The algorithm comparison changes the configured selector while retaining the same workload and receiver lifecycle.

3. Parameters and why each one is needed

TABLE 1. Parameter purpose and literature basis

Parameter	Plain meaning	Reason and paper connection
SPI	How serious does the visible situation appear?	Adds an image-level assessment. CrisisMMD and MEDIC motivate incident-image analysis [5], [6]. Our four urgency levels are project-defined.
Relevance	Does this image match the device's mission?	CLIP compares image and task-text representations [2]. The same image can matter differently for different missions.
Novelty	Does this look different from this sender's recent images?	Separates appearance change from repetition. Content and age are also distinct in semantic freshness work [12]. This is an appearance proxy, not verified new knowledge.
Scene evidence	What configured objects or regions were detected?	YOLO-World and Grounding DINO motivate text-conditioned detection [3], [4]. Counts alone do not establish urgency.
Timing and status	When was it captured or received, and which measurements exist?	Prevents an old image, a failed detector and an empty scene being treated as the same case.

Mission relevance with an example

A drone inspecting road blockage and a camera recording household inventory have different missions. An image of a damaged road may match the first mission strongly and the second weakly. We encode the image and the supported mission description using CLIP, normalize both vectors, and calculate their cosine similarity [2]. A similarity of 0.6 is a model score. It is not a 60 percent probability that the image is useful.

Appearance novelty with an example

The sender compares the current image with its stored images for the same device and mission. If the closest stored image has cosine similarity 0.8, novelty is calculated as one minus 0.8, divided by two, giving 0.1. An identical representation gives zero. The sender queries history before adding the current image, so an image cannot compare against itself.

The executable default keeps 64 recent sender entries unless another capacity is supplied. Empty history has an explicit cold-start flag. Sender history records offered observations, while receiver history records processed outcomes. They serve different purposes. A new viewpoint can look novel without adding a new fact; a small important change can remain visually similar.

4. Scene extraction and representation

What the detector measures

We use YOLO-World with a fixed category list [3]. For each supported category, the extractor records the number of retained detections and the largest bounding rectangle as a fraction of the image. In a 400 by 200 image, a 100 by 80 rectangle occupies 8000 of 80000 pixels, giving an area fraction of 0.1. This is rectangle size in the image, not the object's physical size or distance.

The vocabulary has 93 entries: 80 COCO categories, five additional object categories and eight regions. The added objects are building, bridge, utility pole, traffic sign and animal. The regions are road, water, debris, smoke, flame, snow, ground and grass. Counts apply to 85 object categories. Region-count fields remain unavailable because several water detections do not mean several independent floods.

Why YOLO-World was retained

TABLE 2. Detector-call measurements on the same 300 MEDIC images

Operating point	Median call time	Mean boxes	Zero-detection images
YOLO, confidence 0.25, IoU 0.5, size 640	19.00 ms	1.99	40.0%
Grounding DINO, box 0.35, text 0.25, IoU 0.5	368.69 ms	3.47	13.7%

Operating point	Median call time	Mean boxes	Zero-detection images
Grounding DINO, box 0.35, text 0.35, IoU 0.3	369.64 ms	3.52	12.7%

The detector comparison used 16 nouns. It had no reference boxes, so more detections cannot be called better recall. YOLO had lower measured call time and was retained as the lower-cost baseline. These timings do not establish the speed or accuracy of the later 93-class setup. Grounding DINO provides a different text-grounded detection mechanism [4].

Features and bytes are different

The evaluated scheduler uses a 387-entry base vector containing scene measurements, SPI, relevance, novelty and availability information. This model-input dimension is not a transmitted byte count. A compact packet must separately specify its field widths, quantization and compatibility profile.

5. Compact header packet design

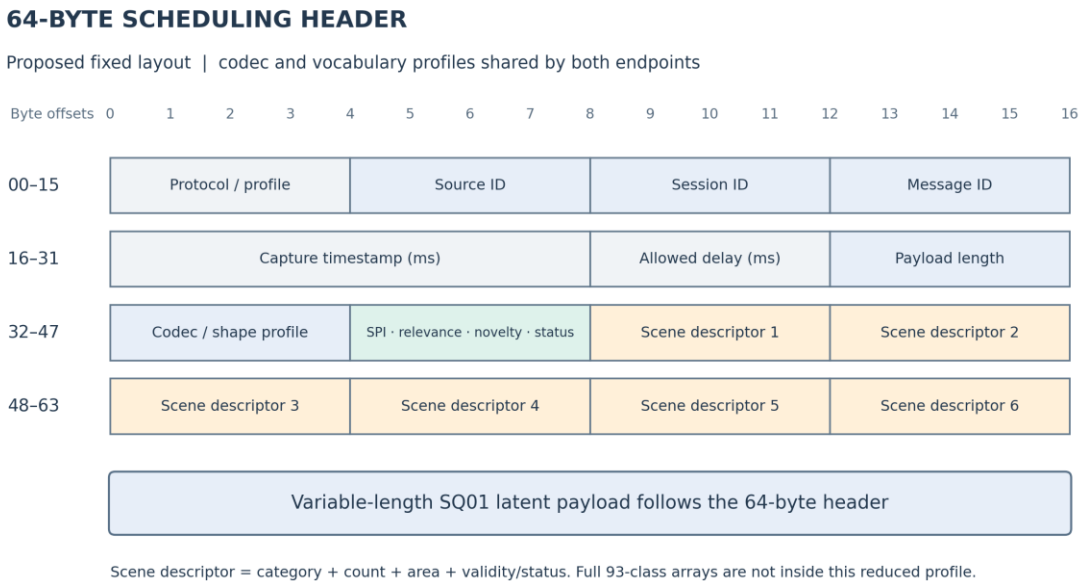


Fig. 3. Proposed 64-byte fixed scheduling header followed by the variable-length SQ01 latent payload. Byte offsets specify the header layout. The six scene descriptors are a reduced profile, not the full scene representation used for the reported comparisons.

The proposed packet uses a fixed 64-byte scheduling header. It carries protocol and profile information, message identity, capture timing, payload description, quantized semantic scores and a bounded scene description. The variable-length SQ-GAN payload follows the header. The 64-byte figure excludes transport framing and the latent payload.

TABLE 3. Proposed 64-byte header allocation

Byte offsets	Field group	Bytes
0–3	Version, header length and profile identifier	4
4–15	Source, session and message identifiers	12
16–27	Capture timestamp and allowed delay, in ms	12
28–35	Payload length and codec/shape profile	8
36–39	SPI, relevance, novelty and status	4
40–63	Up to six compact scene descriptors	24
Total	Fixed scheduling header	64

Each scene descriptor contains a category identifier, quantized count, quantized area and availability/status information. Both endpoints must share the profile's vocabulary, field interpretation and quantization. Unknown or incompatible profiles are rejected. The reduced scene block omits categories beyond the selected descriptor slots; count clipping and area quantization must remain explicit.

The 64-byte layout is a design specification. Reported scheduler scores retain the existing full scene features; they do not establish equivalent performance with this reduced header.

6. Images and SPI classification

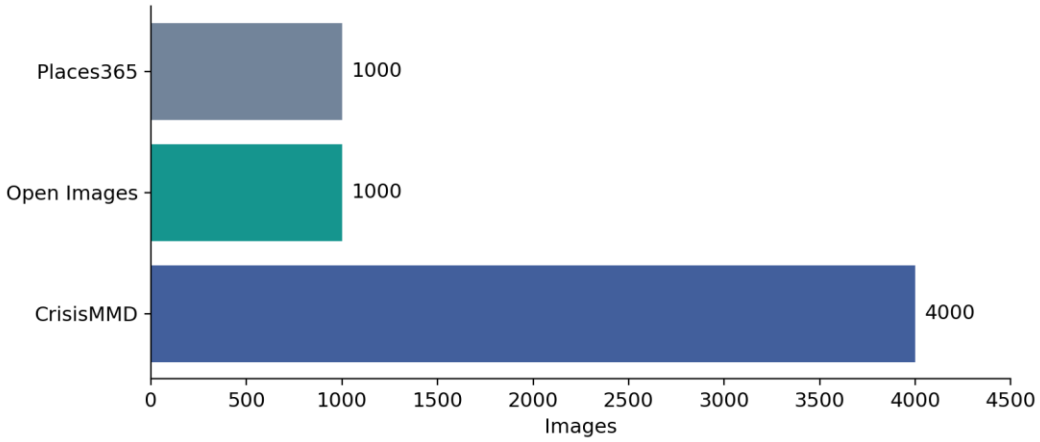


Fig. 4. Image sources in the 6000-image dataset. CrisisMMD supplies the crisis-image labeling cohort; Open Images and Places365 broaden the image pool.

Which images were used and why

The image pool contains 4000 CrisisMMD images, 1000 Open Images images and 1000 Places365 images. The 6000 files contain 5952 distinct raw images. Earlier parameter development used 300 images, including 220 LADI-v2 and 80 MEDIC examples. These stages are separate datasets, not extra independent test images.

CrisisMMD contains disaster-related social-media images and associated humanitarian annotations [5]. MEDIC combines disaster-image sources, including overlapping CrisisMMD material [6]. Such data was suitable for an initial incident-focused SPI task. Its native annotations describe damage, objects or humanitarian categories. They do not directly provide our four intervention-need labels.

Restricting SPI training to crisis imagery limits the claim. A classifier trained there may make errors on ordinary indoor scenes, screenshots or unfamiliar events. Adding everyday images to the receiver dataset broadens inputs but does not prove the classifier generalized correctly to them.

Why four classes

TABLE 4. The four SPI levels

Class	Meaning in this project	Example boundary
Low	No visible problem requiring intervention, or routine content.	An ordinary street scene does not become urgent merely because people are present.
Medium	A visible issue needing attention without clear immediate severe danger.	A maintenance defect can need correction without an immediate emergency.
High	Serious visible damage or disruption supporting prompt attention.	Severe damage does not by itself prove a person is in immediate danger.
Critical	Visible evidence supporting immediate severe intervention need.	The label must not invent hidden victims or an unseen imminent collapse.

Four levels distinguish routine content, nonurgent issues, serious incidents and immediate severe need. Binary incident detection would merge several of these cases. Four levels also keep the classifier output small. The levels are a chosen annotation rubric, not universal emergency categories established by a paper. An unreadable or unsupported image has an unavailable assessment rather than automatically becoming Low.

7. Labeling and training the SPI model

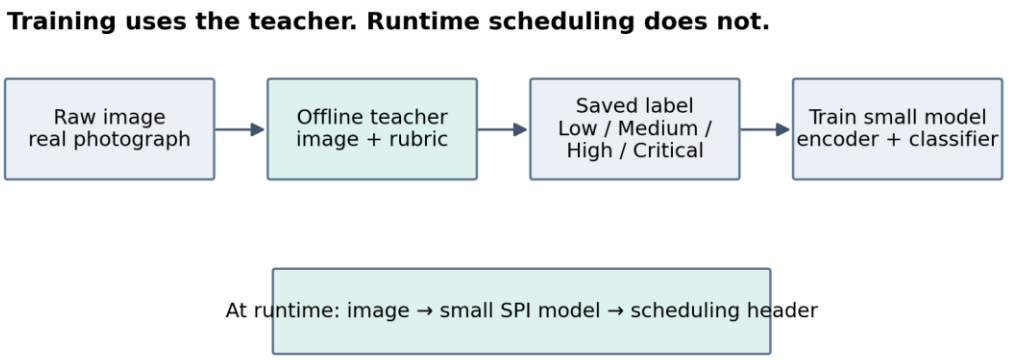


Fig. 5. SPI training and runtime are separate. A VLM supplies offline labels. A smaller image model predicts SPI during parameter extraction.

How labels were obtained

A vision-language model sees an image and a prompt defining visible evidence, the four classes and the required output structure. The system saves the image identity, raw response, parsed label and teacher configuration. Qwen2.5-VL provides the multimodal model foundation [7]. A well-formed response is usable data, but is not proof that the judgment is correct.

The 4000-image labeling file combines 2540 earlier checkpoint assessments and 1460 continuation assessments. The continuation used a pinned Qwen2.5-VL-7B-Instruct checkpoint with thinking disabled. There were 3933 available labels and 67 unavailable assessments. Because the protocols were mixed, this is not described as a single-teacher, single-prompt dataset.

Earlier teacher attempts had malformed responses and unsupported visual claims, including dam or bridge details. Strict parsing detects format failures; it cannot detect every hallucination. The student learns from these AI-generated labels, so its reported accuracy measures agreement with them rather than independently verified real-world urgency. This is supervised classification from pseudo-labels, not automatically reinforcement learning from AI feedback.

TABLE 5. SPI class support after unavailable labels were excluded

Class	Training	Validation	Test	Total
Low	782	419	273	1474
Medium	339	149	200	688
High	899	445	325	1669
Critical	48	24	30	102
Total	2068	1037	828	3933

From an image to a class

The retained SPI pipeline first preprocesses the image, then uses CLIP ViT-B/32 to obtain a 512-value image representation [2]. A logistic-regression head multiplies those values by learned weights and produces four class scores. The head has 2052 scalar coefficients and intercepts. The image encoder accounts for most of the pipeline cost. The CLIP representation used for these predictions is different from the reconstruction codec's latent payload.

8. SPI model comparisons and results

What was compared

The broader study compared four frozen feature encoders: CLIP, SigLIP2, DINO and ConvNeXtV2 [2], [8], [9], [10]. Frozen means their pretrained weights were reused rather than retraining the entire encoder. The saved encoder identity determines which DINO model or fallback was actually used.

The twelve heads were logistic regression, ridge, linear SVM, RBF-SVM, k-nearest neighbors, random forest, extra trees, histogram boosting, MLP, mean-MLP, gated attention MIL and single-query cross-attention. Linear heads test whether the existing features already separate the labels. Nonlinear heads allow more complex boundaries. Token heads examine patch-level features instead of relying only on one pooled vector. Attention-based multiple-instance learning motivates weighted aggregation [11].

The classifier used for the dataset

CLIP with class-balanced logistic regression was fitted on 2068 training examples. Three regularization values, 0.1, 1 and 10, were compared on 1037 validation examples. The selected value was 10. The resulting model was evaluated on the saved 828-image test split and then used to populate SPI predictions in the image dataset.

TABLE 6. SPI classifier results against AI-generated labels

Metric	Result
Validation macro-F1	0.60245
Test macro-F1	0.59997
Test accuracy	64.86%
Low recall	79.49%
Medium recall	36.00%
High recall	69.54%
Critical recall	73.33%

Accuracy counts all correct predictions together. Recall asks how many images of one class were recognized correctly. Macro-F1 gives each class equal influence, so the common classes cannot hide every minority-class error. Medium recall of 36 percent means 72 of its 200 test examples were recognized as Medium.

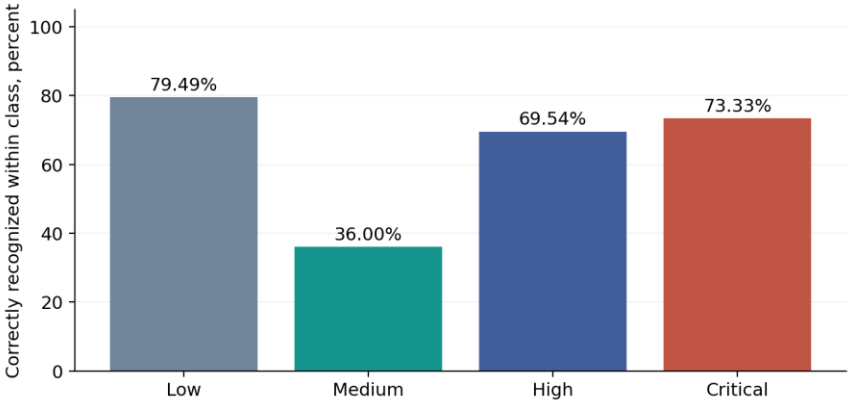


Fig. 6. Class recall for the SPI classifier. Medium was the weakest class. These values measure agreement with pseudo-labels, not human-verified urgency.

9. The waiting budget explained

Waiting-time budgets

In this experiment, a waiting budget means how long a message is allowed to wait before processing should start. It does not force the message to wait that long. A Critical image can start immediately. It also does not mean that its decoder takes 2 ms to run. Every job takes 1 ms in the constant-service control, regardless of class.

TABLE 7. Chosen waiting targets for the scheduling experiment

Reference class	Target waiting time	Meaning
Critical	2 ms	Waiting becomes late after 2 ms.
High	4 ms	Waiting becomes late after 4 ms.
Medium	8 ms	Waiting becomes late after 8 ms.
Low	16 ms	Waiting becomes late after 16 ms.

The numbers 2, 4, 8 and 16 were chosen as experimental settings to make postponing important images more costly. They are not emergency-service standards and were not discovered by the VLM. Using them lets every policy face the same scoring rule. Missing urgency references cannot be filled by simply assigning Low.

Waiting-cost function

A short wait is usually less serious than a long wait. The same wait also matters more when the image has a shorter target. First divide the actual wait by the target. After the target is exceeded, add the excess fraction again. The result is called a waiting penalty. Smaller is better.

For a wait w and a target b, penalty = w/b + max of zero and w/b minus one. Before the target, the penalty rises steadily. After the target, it rises twice as fast. At exactly the target, the penalty is 1.

TABLE 8. Worked examples for the same scoring rule

Image	Wait	Calculation	Penalty
Critical, target 2	1	1/2 + 0	0.50
Critical, target 2	2	2/2 + 0	1.00
Critical, target 2	4	4/2 + (4/2 − 1)	3.00
Low, target 16	4	4/16 + 0	0.25

Thus a 4-ms wait costs 3 for a Critical image and 0.25 for a Low image. This is how the test expresses that delaying the Critical image is more costly. The penalty is dimensionless. A score of 3 does not mean a 3 ms delay.

10. How a complete schedule receives its score

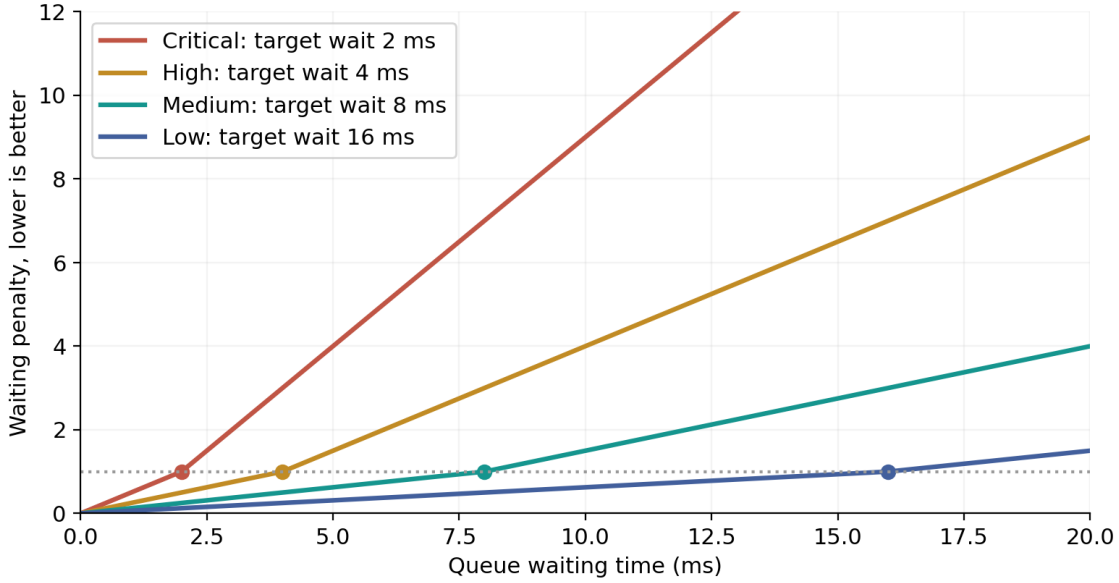


Fig. 7. The waiting-penalty curves for the chosen targets. Each dot marks the target wait, where penalty equals 1. Critical waiting accumulates penalty fastest.

Worked scheduling example

Suppose Routine, High and Critical images all become ready at time 0, and each takes 1 ms to process. Assume the Routine image arrived first among that tied group. FIFO processes Routine, then High, then Critical. Their waits are 0, 1 and 2. Their penalties are 0, 0.25 and 1, giving a mean of 0.4167.

SPI-first processes Critical, then High, then Routine. Their waits are 0, 1 and 2, but the penalties are now 0, 0.25 and 0.125. The mean is 0.125. Both policies processed three images in 3 ms. SPI-first received a better score because it placed the costly wait on the less urgent image. This example assumes correct priority labels and is illustrative, not a measured run.

Images left in the queue still count

The experiment adds the penalty of every assessable offered image and divides by the number of those images. An image that starts processing stops accumulating waiting penalty. Its existing penalty remains in the total. An image still waiting when the episode ends contributes its elapsed wait up to that endpoint. Otherwise a policy could appear good by serving only a few easy images and ignoring the rest.

How the score is used for learning

Supervised schedulers learn which candidate would cause the most extra penalty if it waited one additional 1 ms service interval. Reinforcement-learning agents receive a negative reward when total waiting penalty increases. The stored reward divides that increase by 100 to control its numeric scale. This scaling is a training setting, not another waiting target.

Reference priority is kept separate from runtime inputs. A policy acts on predicted SPI and permitted context; the evaluator scores it using the saved reference. The current references are silver labels, meaning model-generated judgments rather than independently verified operational urgency.

11. Traditional and fixed-rule scheduling algorithms

These methods do not train a queue-selection model. Each follows a fixed rule. Some ignore image meaning, while others use the extracted semantic parameters. All choose from the same ready queue and obey the same one-job-at-a-time rule.

TABLE 9. The eight fixed-rule policies evaluated

Policy	How it chooses	What it tests
FIFO	Choose the earliest arrival.	Whether arrival order alone is sufficient.
Round robin	Rotate service among senders.	Fairness between devices.
Random	Choose a random ready message using a saved seed.	A chance-based control.
SPI-first	Choose the highest predicted priority; break ties by arrival.	A simple importance-aware rule.
Relevance-first	Choose the strongest mission match.	Whether task alignment alone gives a useful order.
Novelty-first	Choose the largest appearance change.	Whether visual difference alone gives a useful order.
SPI then novelty	Choose priority first and novelty within the class.	Whether repetition helps resolve priority ties.
Predicted EDF	Choose the earliest arrival-plus-predicted-budget deadline.	A time-aware rule based on predicted classes.

12. Learned scheduling algorithms

A learned scheduler receives numeric inputs and produces a score for each waiting candidate. The receiver selects the candidate with the highest score. Different models learn that score in different ways. The architecture describes how the score is computed; the loss or reward describes how training changes it.

TABLE 10. Supervised methods and their plain-language roles

Method	How it learns or scores
Linear choice	Adds learned weighted contributions from the inputs. It is the simplest trained scorer.
MLP choice	Uses a small neural network to learn nonlinear combinations and select a candidate.
Delay regression	Predicts the extra waiting cost caused by postponement. Huber loss reduces the influence of large individual errors.
Priority classifier	Predicts four reference classes and uses them for waiting-sensitive selection.
RankNet	Learns which member of a candidate pair should be ahead [13].
LambdaMART	Combines many small decision trees to learn useful ordering [14].
Deep Sets RankNet	Scores a candidate using its own fields plus pooled queue and history information [15].
Tie-aware listwise	Learns an ordering across several candidates while respecting equal-cost cases. The implementation adapts listwise learning [17].
Attention set ranker	Lets candidate representations exchange information before ranking. It is an adaptation of set attention [16].
Temporal Deep Sets	Adds a GRU that reads up to three recent observations in order before scoring.

13. Reinforcement-learning schedulers

TABLE 11. The four reinforcement-learning methods

Method	Plain-language explanation
PPO	Learns a probability for each available choice and adjusts the policy using accumulated reward [20].
Double DQN	Learns the value of choosing a candidate. Separate networks choose and evaluate the next action during training [18].
Double DQN with prioritized replay	Revisits stored training transitions more often when their prediction errors are larger [19]. This replay priority is not operational image priority.
Ranker-initialized PPO	Starts PPO from an already trained ranker instead of a fresh actor, then updates it using queue rewards.

Fourteen scheduler methods completed development evaluation. Compute, GPU memory, storage and runtime limited the broader comparison and repetition. These results do not establish convergence or repeated-seed stability. The separate SPI comparison ended after storage exhaustion.

14. How the scheduling comparison was run

A shared workload for a fair comparison

The simulator saves message arrivals, processing times and possible outcomes before comparing policies. Each policy receives the same realization. Thus a policy is not allowed a different, easier set of arrivals just because it is a different algorithm. Only complete messages become eligible, and later arrivals remain unavailable until their arrival time.

Development validation includes device populations of 4, 12, 50, 60 and 100. Fifteen mission templates represent phones, inspection and search drones, vehicle cameras, CCTV, industrial robots and other sources. A device mission supplies context, not an automatic priority. A normal phone can capture a serious incident; an inspection drone can send a routine repeat.

Matched workload conditions

One test keeps total offered load fixed while changing the number of devices. Another keeps each device's rate fixed, so adding devices also adds traffic. Bursts concentrate arrivals in time. Joining and pausing devices alter achieved traffic. These controls separate the effect of more senders from the effect of more messages.

What the reported cohort covers

The broad pool contains 6000 images. The saved receiver report lists 5376 training, 417 validation and 207 test records for that pool, with a separate 3915-record cohort having assessable delay references. Those counts differ from the SPI classifier's 2068/1037/828 split because the experiments use different eligible records and roles. They must not be presented as one split.

The following comparison uses saved development results, not the protected final scheduler test. Model-generated priority references define the assessable urgency scope. Simulated times and mixed image streams are not live recordings of independent emergency incidents. The reported numeric differences describe these runs; they are not confidence intervals or evidence of deployment safety.

15. Direct comparison of all evaluated algorithms

TABLE 12. All eight fixed rules and fourteen learned schedulers on the development comparison

Algorithm	Type	Penalty ↓	Change vs FIFO
LambdaMART	Learned	3.2183	+44.72%
Delay regression	Learned	3.3573	+42.33%
RankNet	Learned	3.3642	+42.22%
SPI-first	Semantic rule	3.3741	+42.05%
Deep Sets RankNet	Learned	3.3819	+41.91%
Tie-aware listwise	Learned	3.3859	+41.84%
SPI then novelty	Semantic rule	3.4349	+41.00%
Temporal Deep Sets	Learned	3.4480	+40.78%
Attention set ranker	Learned	3.4520	+40.71%
MLP choice	Learned	3.5675	+38.72%
Ranker + PPO	Learned	3.6375	+37.52%
Priority classifier	Learned	3.8271	+34.26%
Linear choice	Learned	4.3392	+25.47%
PPO	Learned	4.5935	+21.10%
Double DQN	Learned	4.7099	+19.10%
Relevance-first	Semantic rule	4.9203	+15.49%
Predicted EDF	Semantic rule	5.0266	+13.66%
Double DQN + replay priority	Learned	5.1757	+11.10%
Round robin	Traditional	5.8059	+0.28%
FIFO	Traditional	5.8220	+0.00%
Random	Traditional	5.9196	-1.68%
Novelty-first	Semantic rule	6.1917	-6.35%

LambdaMART recorded the lowest penalty, 3.2183. FIFO recorded 5.8220 and round robin recorded 5.8059. SPI-first recorded 3.3741. Thus the best learned model improved on FIFO by 44.72 percent, while the fixed SPI-first

rule improved on FIFO by 42.05 percent. The additional reduction from SPI-first to LambdaMART was 4.62 percent. These percentages describe waiting penalty, not time saved in milliseconds.

16. What changes when priority is used

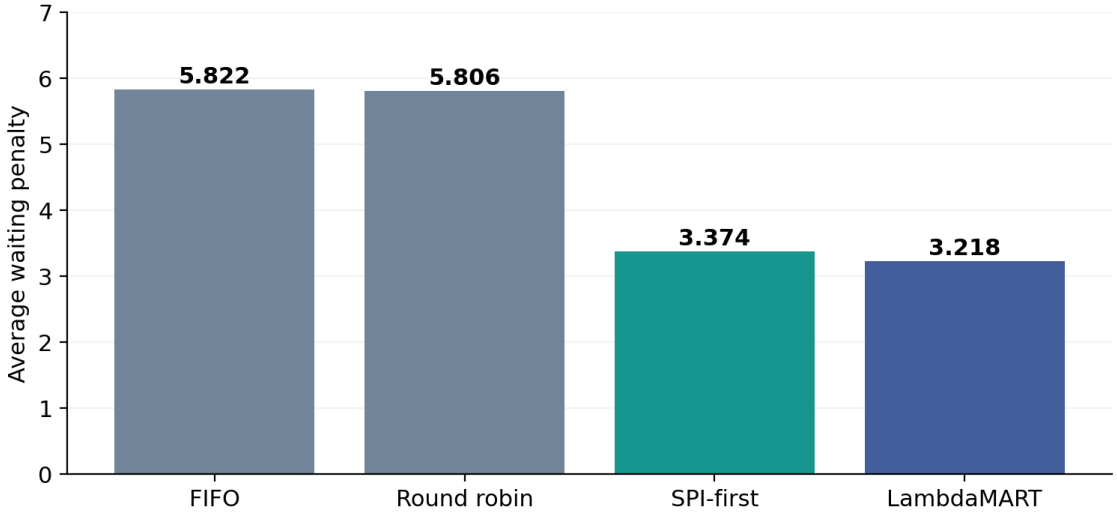


Fig. 8. Four representative methods on the same development comparison. Most of the observed FIFO-to-LambdaMART reduction was already achieved by the simple SPI-first rule.

FIFO and round robin produced similar penalties. SPI-first was the strongest fixed semantic rule. LambdaMART, delay regression and RankNet were the only learned methods that numerically improved on SPI-first. Temporal and attention models did not beat that control, and ranker-initialized PPO was the best of the four RL variants. These are results for the recorded workload and objective, not a universal ranking of algorithm families.

The objective charges waiting according to reference priority and elapsed wait. It does not independently reward every mission or novelty distinction within a class. The table therefore measures priority-sensitive waiting rather than complete semantic understanding.

17. Which classes waited less

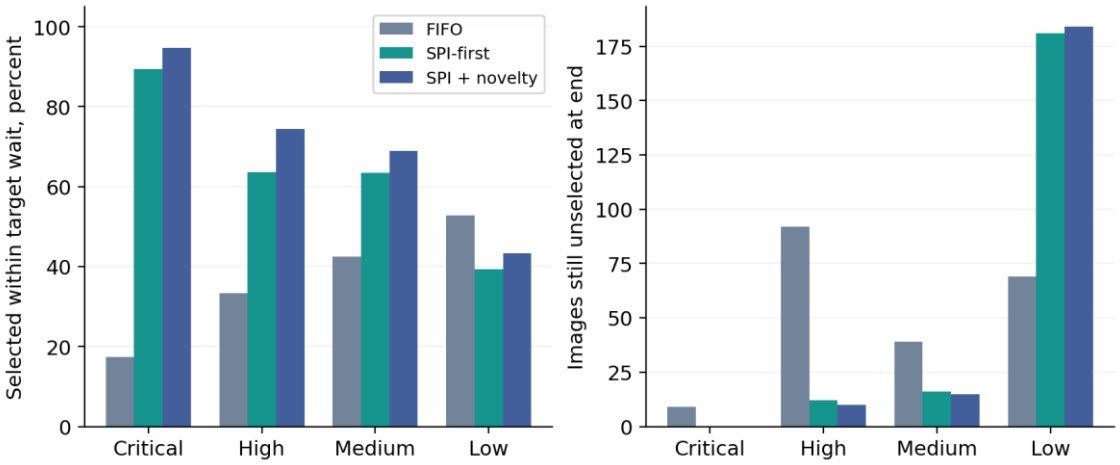


Fig. 9. Left: the percentage selected within the class target. Right: messages still unselected when the episode ended. Both views matter because improving urgent service can increase waiting elsewhere.

TABLE 13. FIFO and SPI-first on the same 15 validation episodes

Policy	Class	Offered	Unselected	Selected mean wait	Within target
FIFO	Critical	75	9	21.441	17.3%
SPI-first	Critical	75	0	2.929	89.3%

Policy	Class	Offered	Unselected	Selected mean wait	Within target
FIFO	High	1087	92	22.099	33.3%
SPI-first	High	1087	12	8.314	63.6%
FIFO	Medium	435	39	20.884	42.5%
SPI-first	Medium	435	16	15.005	63.4%
FIFO	Low	846	69	19.282	52.8%
SPI-first	Low	846	181	34.582	39.4%

For Critical images, SPI-first left none unselected and had a selected mean wait of 2.929 ms, compared with FIFO's 21.441. However, it left 181 Low images unselected, compared with FIFO's 69. Prioritization therefore changed who received service sooner; it did not make every class better at the same time.

Selected mean wait includes only messages that started processing. It must be read with unselected counts. The within-target percentage uses the evaluator's eligible follow-up observations. Those denominators differ, so a rounded percentage should not be reconstructed simply by dividing selected messages by all offered messages. Class-level learned-model outcomes are not supplied by the retrieved aggregate table, so the report does not assign LambdaMART an invented Critical-class benefit.

18. Computational cost and experiment scope

Where resources were spent

The expensive stages were not limited to scheduler scoring. They included VLM labeling, image-encoder feature extraction, storage of patch-level features, classifier searches and RL interactions. A 49 by 768 CLIP token array uses 150528 bytes per image in float32. Storing it for 3933 images takes about 565 MiB before compression and metadata. Multiple feature banks and checkpoints multiply that cost.

The broader SPI comparison completed twelve original-setting heads each for CLIP, SigLIP2 and DINO, for 36 completed candidate results. ConvNeXtV2 feature extraction stopped when storage filled after about 97 minutes. Other balancing regimes and the complete final selection did not finish. The best development candidate among the 36 was CLIP with single-query cross-attention, macro-F1 0.63365. This is a different candidate and evaluation role from the dataset classifier's test macro-F1 0.59997.

Measured SPI inference cost

TABLE 14. Raw-image-to-SPI profile of CLIP plus logistic regression

Measurement	CPU	GPU
Median call	104.74 ms	28.56 ms
95th percentile	112.46 ms	38.84 ms
Timed calls	30	100

The profile used the same ten development images, ten warmups, three CPU repeats per image and ten synchronized GPU repeats. It covers the SPI path, not the full detector, mission, novelty and serialization pipeline. The older detector timings used another scope, so their percentile values cannot simply be added to these values to create a measured full-header percentile.

What the evidence supports

The experiments support a completed extraction dataset, measured pseudo-label agreement for SPI, and development comparisons of fixed and learned schedulers. They do not establish general emergency understanding, exact image recovery or measured end-to-end radio latency. The SPI test cohort had already been exposed during development. The protected scheduler final test remained unopened in the supplied records. The delay references are model-generated, and queue variations from one seed are not independent disaster events.

Conclusion

Semantic priority changed processing order enough to reduce the recorded waiting penalty substantially compared with FIFO. LambdaMART gave the lowest score among the 22 compared methods, but SPI-first captured most of the observed improvement with a fixed rule. The results therefore distinguish two findings: using priority information helped strongly in this experiment, while the extra gain from learning was smaller. The report states both results together, with the waiting rule, workload and resource limits that define them.

Appendix A. Complete SPI encoder comparison

The values below are development macro-F1, not final test scores. Macro-F1 averages the four class F1 scores equally. All 36 rows completed in the original-setting comparison; ConvNeXtV2 did not complete its feature extraction. The same numbers are used in the source completed-comparison CSV.

TABLE 15. All completed encoder and classifier pairs

Classifier head	CLIP	SigLIP2	DINO
logistic	0.4398	0.4401	0.5317
ridge	0.4517	0.4624	0.5249
linear_svm	0.4862	0.5490	0.5783
rbf_svm	0.6141	0.6003	0.5796
knn	0.5832	0.6031	0.5686
random_forest	0.4193	0.4250	0.4130
extra_trees	0.4386	0.4424	0.4364
hist_boost	0.5386	0.5320	0.5092
mlp	0.5896	0.6152	0.5835
mean_mlp	0.6120	0.6031	0.5592
gated_abmil	0.6079	0.5717	0.5661
single_query_cross_attention	0.6337	0.6263	0.5710

Every head requires the same encoder and preprocessing used during fitting. The encoder comparison is separate from the scheduler comparison.

References

[1] Bourtsoulatze, Kurka and Gunduz. Deep Joint Source-Channel Coding for Wireless Image Transmission. 2019. Online source

[2] Radford et al. Learning Transferable Visual Models From Natural Language Supervision. ICML 2021. Online source

[3] Cheng et al. YOLO-World Real-Time Open-Vocabulary Object Detection. CVPR 2024. Online source

[4] Liu et al. Grounding DINO. 2023, revised 2024. Online source

[5] Alam, Ofli and Imran. CrisisMMD Multimodal Twitter Datasets from Natural Disasters. ICWSM 2018. Online source

[6] MEDIC dataset and documented annotation tasks. Online source

[7] Bai et al. Qwen2.5-VL Technical Report. 2025. Online source

[8] Tschannen et al. SigLIP 2. 2025. Online source

[9] Oquab et al. DINOv2 Learning Robust Visual Features without Supervision. 2023. Online source

[10] Woo et al. ConvNeXt V2. CVPR 2023. Online source

[11] Ilse, Tomczak and Welling. Attention-based Deep Multiple Instance Learning. ICML 2018. Online source

[12] Maatouk, Assaad and Ephremides. The Age of Incorrect Information. 2020, revised 2022. Online source

[13] Burges et al. Learning to Rank using Gradient Descent. ICML 2005. Online source

[14] Burges. From RankNet to LambdaRank to LambdaMART An Overview. 2010. Online source

[15] Zaheer et al. Deep Sets. NeurIPS 2017. Online source
[16] Lee et al. Set Transformer. ICML 2019. Online source
[17] Xia et al. Listwise Approach to Learning to Rank Theory and Algorithm. ICML 2008. Online source
[18] van Hasselt, Guez and Silver. Deep Reinforcement Learning with Double Q-learning. AAAI 2016. Online source
[19] Schaul et al. Prioritized Experience Replay. ICLR 2016. Online source
[20] Schulman et al. Proximal Policy Optimization Algorithms. 2017. Online source
[21] F. Pezone, S. Barbarossa and G. Caire. SQ-GAN: Semantic Image Communications Using Masked Vector Quantization. IEEE TCCN, 2025. Online source